

Temperature scaling and multi-sample evaluation for CoBit on verifiable tasks

A research memo and handover

csic47, August 2026 · github.com/GBATZOLIS/BitstreamDiffusion, branch `tasks/fkc-temperature`

Why we started

The CoBit paper makes one honest concession. On grade-school maths via executable code (TinyGSM→GSM8K) CoBit reaches about 29% accuracy, which beats every continuous diffusion or flow model by a wide margin and beats MDLM and Duo when those are sampled at standard temperature. But when MDLM and Duo are allowed their tuned low-temperature decoding ($T \approx 0.1$) they reach roughly 33% and 36%, and we fall behind. We wrote that this points to “a missing sharpening mechanism rather than a limitation of the bitstream representation”, and left it as future work. This memo reports what happened when we tried to build that mechanism, and a second investigation that grew out of it.

The difficulty is that low-temperature decoding has no obvious analogue for us. MDLM and Duo predict a categorical distribution over tokens and can raise its logits to a power. CoBit denoises a continuous bitstream; there is no categorical distribution to sharpen at sampling time. The naive substitute — sharpening the per-bit Bernoulli posterior — was tried earlier on Sudoku and collapses below about $T=0.25$, because sharpening bits independently drives samples off the valid-codeword manifold.

Feynman-Kac correctors offered a principled alternative. Instead of deforming the per-step distribution and hoping, one runs several particles, weights them by an importance weight derived from the tempered target, and resamples. With enough particles the population is distributed as p^β exactly. That is attractive precisely because it is not a heuristic: it states which distribution you are sampling from, and the correction is what makes the statement true. We had already built the machinery for Sudoku, where it did not help. The open question was whether GSM8K — a task with a genuine notion of correctness that the model might assign higher likelihood — would behave differently.

What we found

The short version: temperature scaling did not deliver; the mechanism we thought we were studying turned out not to be the mechanism actually operating; and the most useful result came from something we were not looking for.

The promising phase, and why it misled us

We first swept β at 512 integration steps with 32 particles. Per-particle accuracy rose monotonically, from 29.6% at $\beta=1.01$ to 32.1% at $\beta=1.08$ and 34.0% at $\beta=1.15$. No individual cell was significant against the control, but the trend across eight cells was (slope 95% CI [0.085, 0.705], $p \approx 0.007$ against zero slope). Diversity fell as accuracy rose, exactly as a temperature should behave. For a while this looked like the result we wanted.

Two observations dismantled it.

First, accuracy kept climbing well past the point where the particle cloud had collapsed. By $\beta=1.15$ the effective sample size was 1.0 out of 32 and the average number of distinct answers per problem was 1.18 — all thirty-two particles were returning the same thing. A Sequential Monte Carlo method whose population has degenerated to a point mass is not performing inference; whatever was improving accuracy, it was not the corrector.

Second, and decisively, we reran the sweep with **a single particle**, no weights and no resampling, so β could act only through the drift. Accuracy still rose: 28.5% at the control, 30.9% at $\beta=1.10$, 32.8% at $\beta=1.20$. Roughly four of the five points of apparent gain were reproducible with no Feynman-Kac machinery at all. The corrector itself contributed perhaps one to two points, never significantly.

The reason is visible in the code. β enters in two places, not one. It multiplies the importance weight, which is the intended tempering channel — but it also multiplies the entire probability-flow drift ($x = x + h \cdot \beta \cdot d$). At a coarse step count the integrator systematically undershoots, and inflating the drift by five or ten percent partially compensates for the discretisation error. That is a step-size correction wearing a temperature's clothing.

The test that settled it

If β were genuinely a temperature, its benefit should not depend on how finely we integrate. If it compensates truncation error, it should help at coarse steps and hurt at fine ones, because the same inflation becomes an overshoot once the integrator is accurate.

So we ran the configuration where the corrector is best behaved: 1024 steps, 64 particles, $\beta=1.05$. This is the only setting in the whole study where the population stayed healthy — an effective sample size of about 11 out of 64, and seven distinct answers per problem, rather than the near-total collapse seen elsewhere. Against a

matched single-sample control on the same problems, tempering **lost 5.65 points** (95% CI $[-10.62, -0.88]$). The sign flipped, as the step-size hypothesis predicts, and this time the effect was significant.

Meanwhile the plain step count mattered more than anything we were tuning. Going from 512 to 1024 steps, with one sample and no temperature, gains 7.03 points (95% CI $[+1.95, +12.50]$). Our best tempered configuration at 512 steps — 34.0%, using 32 particles — is still worse than simply integrating properly with a single sample (35.5%). An uncomfortable comparison, but a clarifying one.

We also tested the one particle-free mechanism we had: a local score-temperature rescaling the score by $\kappa(\sigma)=(v+\sigma^2)/(\tau v+\sigma^2)$, so that it sharpens only late in the trajectory. On Sudoku this had given a small, monotone, cliff-free gain of about two points. On GSM8K it gives +0.8 at $\tau=0.7$, then degrades, and at $\tau=0.3$ the model produces no executable programs at all. It does not transfer.

Where this leaves the sharpening question

Our reading is that β behaves as a step-size correction rather than a temperature, and that none of the sharpening mechanisms available to us closes the gap to tuned low-temperature discrete diffusion. The one lever that clearly works — more integration steps — is not a sharpening mechanism, and the paper already operates at the good end of it.

We would not present this as a settled negative, for the reasons below.

Why we do not fully trust our own conclusion

There is a difference between “temperature scaling does not help CoBit” and “our implementation and application of Feynman-Kac correctors did not help CoBit”, and we cannot presently distinguish them.

We cannot see the quantity that would explain it. The natural diagnostic is whether the FKC weight ranks samples by correctness. If the model’s likelihood does not know which programs are right, concentrating probability mass cannot help, and the negative becomes mechanistic rather than incidental. We do measure this — and in the arm where the gain lives, the measurement is void. The reason is subtle. Resetting log-weights to zero after each resampling event is *correct* SMC: the weights have been consumed by the resampling, and carrying them forward would double-count the evidence. But we record the weight vector at the end of the trajectory, and at the β values of interest resampling fires at essentially every step including the last, so the recorded weights are uniform to eight decimal places ($0.03125 = 1/32$). The diagnostic reads exactly zero signal by construction. In the arms where resampling is disabled and weights survive the full path, a real signal appears: correct programs carry about 7% more accumulated weight than incorrect ones at $\beta=1.08$, and the separation grows with β as the $\frac{1}{2}\beta(\beta-1)$ coefficient predicts. That is weak, but it is not

nothing — and it is the opposite of what we saw on Sudoku. Fixing it needs a non-reset cumulative path potential carried along the ancestry: diagnostic only, incapable of changing results, and the first thing anyone should do before drawing conclusions from this code.

Our particle counts may be hopeless rather than merely small. The FKC log-weight is extensive in the number of unconstrained variables: every free bit contributes. Sudoku has about 356 free bits; GSM8K, at 512 tokens and 16 bits per token, has around 7,200. Weight variance grows accordingly, and the particle count needed to keep a population healthy grows roughly exponentially in that variance. We never exceeded 64 particles. The honest experiment may need hundreds or thousands — a distributed-sampling engineering problem rather than a hyperparameter sweep. Doubling from 32 to 64 changed nothing measurable (-0.06 points), which is consistent either with the effect being absent or with both counts being far below what the problem requires.

The proposal is only approximately right. The EDM-churn proposal is exact at $\beta=1$ but only leading-order for $\beta>1$, because tempering does not commute with the Gaussian churn step; the code emits a warning saying so. Halving the step count makes the approximation worse, which is awkward given that the apparent gains all live at 512 steps. The refinement study the warning asks for was never run.

We barely explored the space. Throughout, the resampling threshold stayed at half the particle count, the policy at ESS-triggered, the self-conditioning policy at “inherit”, and the prior at its default. The alternatives already in the codebase — resampling every active step, the zero and stateless self-conditioning policies, confining resampling to the informative entropy band, the forward-marginal prior — were never tried on GSM8K. Nor did we vary β along the trajectory; every run used a constant.

Three knobs are entangled and were never separated. β , the churn parameter γ , and the step count interact. γ was pinned at 0.41, the EDM stability ceiling, in every single cell, and the effective Langevin strength depends on the product of γ and the step count — so changing the schedule changes stochasticity and resolution together. Our central claim about the sign flip is confounded with having held γ fixed.

The comparison that matters is thin. The experiment isolating whether the corrector earns its keep — corrected versus drift-only at matched β — was run at two β values, one step count, 256 problems. It deserves more than we gave it.

Finally, the two-model classifier-free-guidance variant of FKC was never tested here, though the conditioning-dropout checkpoint it needs already exists on the cluster.

The second investigation: how many samples does the model need?

While setting this up we noticed something about the literature. Every published comparison on these two benchmarks — including the S-FLM paper that defines the

protocol and supplies all our baseline numbers — evaluates with **one sample per problem**. No pass@k, no majority vote, no best-of-n anywhere.

That gap matters for a specific reason. Recent work on reinforcement learning from verifiable rewards suggests RL largely redistributes probability mass among solutions the base model can already produce rather than creating new ones: it raises pass@1 while often lowering pass@k at large k. If so, a model’s pass@k profile measures what post-training could eventually extract from it. Since post-training for diffusion language models so far exists only for masked models, asking how much latent capability a continuous bitstream model holds seemed both answerable and unanswered.

We had a hypothesis, and it was wrong, which is the interesting part. The hypothesis was that low-temperature decoding buys pass@1 by spending diversity, and diversity is what pass@k needs. CoBit gets its accuracy from stochastic churn with no temperature, so we expected the profiles to cross — worse at k=1, better at large k — implying more post-training headroom than the single-sample number suggests.

We measured it with thirty-two samples per problem for every method, on the same 256 randomly chosen test problems, with the same execution sandbox grading every output.

method	pass@1	pass@4	pass@8	pass@32	maj@32
CoBit (no temperature)	29.1%	46.4%	53.5%	66.0%	45.7%
Duo, T=0.1	37.1%	61.4%	70.2%	83.6%	61.3%
MDLM, T=0.1	35.4%	59.1%	67.7%	80.9%	59.4%
MDLM, T=1.0	16.1%	38.4%	49.9%	68.0%	52.3%

The curves do not cross. Duo at its tuned low temperature beats CoBit at every sampling budget and on majority vote. More interestingly, comparing MDLM at the two temperatures shows that sharpening lifts pass@1 from 16.1% to 35.4% *and* pass@32 from 68.0% to 80.9% simultaneously. Low-temperature decoding does not spend headroom; it adds it.

There is a real effect in the predicted direction, but it is relative rather than absolute: sharpening flattens the climb from k=1 to k=32 from a factor of 4.2 to a factor of 2.3. The population does become less diverse — it simply starts from a high enough base that the absolute curve rises everywhere anyway. We had treated the multiplier as the figure of merit when the level is what matters.

Two things are worth keeping. It is a genuine and slightly counterintuitive finding about a metric nobody in this literature reports, and it cuts against the RL intuition we borrowed. And CoBit’s own headroom is substantial in absolute terms — 29.1% to 66.0%, with genuinely independent samples, since we ran them with the corrector disabled. It is simply not competitive with tuned discrete diffusion on this task.

Sudoku is a different picture, and we did not finish it. CoBit is the strongest published model on all three difficulties, and earlier multi-sample runs during the FKC study

suggested majority voting on hard puzzles lands near 91% against a single-sample 65.9%. That gap is large enough to be worth measuring properly, and Sudoku is where CoBit’s case is strongest. We stopped to free the GPUs.

Where we would go next

For a publishable result quickly, the multi-sample evaluation is much closer than the temperature work. It needs the GSM8K table completed — one baseline cell crashed on an integer-overflow bug in the baseline repository’s metrics code, S-FLM was never run, and the numbers cover 256 of the 1,319 test problems — and it needs the Sudoku half, which is where CoBit leads. The infrastructure for all of it exists.

To settle the temperature question properly, we would work in this order. Repair the weight diagnostic first: until then nobody can see whether the model’s likelihood carries any correctness signal, and that single fact determines whether tempering could ever work here. Then push the particle count as far as the hardware allows, since the extensivity argument suggests our counts may be off by orders of magnitude rather than a factor of two. Then grid β against γ and the step count together, since the central claim about the sign flip is confounded by having held γ fixed. Only then revisit proposal accuracy and the untried resampling and self-conditioning policies.

Our prior is that the negative will survive: the drift explanation accounts for the data economically and predicts the sign flip we observed. But it rests on diagnostics we know to be blind and particle counts we suspect are inadequate, and it would be wrong to record it as settled.

Appendix: what is where, and how to run it

Everything is on branch `tasks/fkc-temperature` of `github.com/GBATZOLIS/BitstreamDiffusion`.

The samplers are in `diffusion/continuous/samplers.py` and `diffusion/continuous/smc.py`, with 23 correctness assertions in `tests/test_fkc.py` that should be kept green (`PYTHONPATH=. python tests/test_fkc.py`). The task evaluations are `evaluation/tasks/gsm8k_eval.py` and `evaluation/tasks/sudoku_eval.py`; both support FKC tempering, the score-temperature, and multi-sample metrics. Every FKC cell we ran is tabulated in `results/fkc_temperature_all_cells.csv` — 37 rows giving β , particle count, steps, effective sample size, resampling events, accuracy, majority vote, `pass@k` and diversity — and the `pass@k` results are in `results/gsm8k_passk_shardA/`. The earlier Sudoku study is in `docs/fkc_temperature_findings.md`.

The baselines come from the S-FLM authors’ repository, which also ships MDLM, Duo, FLM and CANDI checkpoints for TinyGSM. Our modifications to it — multi-sample sampling, the unbiased `pass@k` estimator, and a shared grader so every method is scored by identical code — are vendored as a patch in `external/s-flm-patches/`, with a README covering the environment (building flash-attn against a matching CUDA

is the one real obstacle) and the test-set sharding scheme that lets runs on different machines be merged.

A representative command, for thirty-two independent samples per problem:

```
python -m evaluation.tasks.gsm8k_eval \
  --config configs/tasks/tinygsm_bits.py \
  --checkpoint runs/tasks/tinygsm/cobit_raw_binary_bits/checkpoints/step=000425000.pt \
  --sampler_kind fkc_em --proposal edm_churn --churn_gamma 0.41 \
  --beta 1.0 --num_particles 32 --resampling_policy never --final_resample 0 \
  --steps 1024 --limit 1319 --batch_size 2 --ema 1 --seed 42 \
  --sigma_data 0.399844765663147 --out_dir <somewhere>
```

Setting `--beta 1.05 --resampling_policy ess --final_resample 1` turns the corrector on; `--score_temp_tau 0.7` selects the particle-free sharpening instead.

Four practical warnings, each of which cost us time. The `sigma_data` value is **not** stored in these checkpoints, which predate the commit that persists it: pass 0.399844765663147 explicitly, because the configuration default of 0.5 is wrong and fails silently. Do not evaluate on a prefix of the test set — the first 256 problems are about six points easier than the full set and will flatter any method. The GSM8K output filename does not include the checkpoint step, so re-running a different checkpoint without setting `--out_dir` overwrites the previous result. And the baseline samplers default to double precision; we ran everything in single precision for a 2.75× speedup after checking that Duo’s single-sample accuracy stays within its confidence interval (we measure 19.5% against 17.2% published, intervals overlapping), but any comparison must state this.

Two reproduction facts worth recording. The paper’s headline 29.19% reproduces at 29.42%, and belongs to the main-run 425k checkpoint rather than the Isambard one; that artifact had simply never been saved. And the 500k checkpoint gives 29.34%, statistically indistinguishable from 425k, so the training curve has flattened and further training is no longer a lever.